SOFTWARE ENGINEERING DEVELOPMENT STANDARDS

& SECURITY REQUIREMENTS

Java Spring Boot | Python | Go | Vue.js | PHP | Angular

Version 1.0 | 2026

Issued by: Software Engineering Director - Adel Samy

1. Purpose & Scope

This document establishes mandatory development standards for all engineers contributing to software products built with Java Spring Boot, Python, Go, and Vue.js. It is issued by the Software Engineering Director and is effective immediately upon publication.

Compliance with this standard is required for all code entering any environment (development, staging, or production). Non-compliance is a blocker during code review.

🎯 **Primary Goal: Eliminate Critical and Major security vulnerabilities before they reach production.**

2. Severity Classification

All security and quality findings are classified using the following four-tier model. Critical and Major findings must be resolved before merge approval.

Level	Definition & Required Action
🔴 CRITICAL	Remote code execution, authentication bypass, data breach vector, injection flaws. Immediate fix required. PR cannot be merged. Must escalate to Director.
🟠 MAJOR	Privilege escalation, sensitive data exposure, SSRF, insecure deserialization, broken access control. Fix required before merge. No exceptions.
🟡 MINOR	Security misconfiguration, verbose errors in production, deprecated API usage. Fix within current sprint.
🔵 INFO	Best-practice deviations, code-quality recommendations. Address in follow-up ticket.

3. Universal Rules (All Languages)

The following rules apply to Java Spring Boot, Python, Go, and Vue.js equally. Language-specific extensions follow in Section 4.

3.1 Authentication & Authorization

ID	Severity	Area	Requirement / Prohibition
U-A01	🔴 CRITICAL	**AuthN**	Never hard-code credentials, API keys, or secrets in source code or config files committed to version control.
U-A02	🔴 CRITICAL	**AuthZ**	Every API endpoint must enforce authorization. No endpoint may be implicitly public unless explicitly annotated and reviewed.

ID	Severity	Area	Requirement / Prohibition
U-A03	🔴 CRITICAL	Session	Session tokens must be cryptographically random, ≥ 128 bits, and expire after inactivity. Never transmit over HTTP.
U-A04	🟠 MAJOR	AuthN	Implement account lockout or rate limiting after 5 failed authentication attempts.
U-A05	🟠 MAJOR	AuthZ	Enforce the principle of least privilege. Never grant broader permissions than the minimum required.

3.2 Input Validation & Injection Prevention

ID	Severity	Area	Requirement / Prohibition
U-I01	🔴 CRITICAL	SQL/NoSQL	Never concatenate user input into SQL or NoSQL queries. Always use parameterized queries or ORM-level binding.
U-I02	🔴 CRITICAL	Command Injection	Never pass unsanitized user input to OS commands, shell, or process execution functions.
U-I03	🟠 MAJOR	XSS	Sanitize and encode all user-controlled data before rendering in HTML, JSON responses, or log output.
U-I04	🟠 MAJOR	Validation	Validate all inputs server-side. Client-side validation is UX only — never a security control.
U-I05	🟠 MAJOR	File Upload	Validate file type, size, and content. Store uploads outside webroot. Never execute uploaded files.

3.3 Cryptography & Data Protection

ID	Severity	Area	Requirement / Prohibition
U-C01	🔴 CRITICAL	Passwords	Never store passwords in plaintext or with reversible encryption. Use bcrypt, argon2id, or scrypt with appropriate work factor.
U-C02	🔴 CRITICAL	Transport	All external and internal service communication must use TLS 1.2+. Enforce HSTS. No HTTP fallback in production.
U-C03	🟠 MAJOR	Crypto Algorithms	Never use MD5, SHA-1, DES, RC4, or ECB mode for security purposes. Minimum: AES-256, SHA-256, RSA-2048.
U-C04	🟠 MAJOR	PII / Sensitive Data	Encrypt PII at rest. Apply data minimization. Log only what is necessary — never log passwords, tokens, or card numbers.

3.4 Dependency & Supply Chain Management

ID	Severity	Area	Requirement / Prohibition
U-D01	🔴 CRITICAL	Known CVEs	No dependency with a CRITICAL CVE may be shipped. Run OWASP Dependency-Check or Snyk in the CI pipeline.
U-D02	🟠 MAJOR	Known CVEs	No dependency with a MAJOR CVE may be shipped without a documented, Director-approved exception.
U-D03	🟠 MAJOR	Pinning	Pin all dependency versions in lock files (pom.xml, requirements.txt, go.sum, package-lock.json). No wildcard versions.
U-D04	🟠 MAJOR	License	Review licenses of new dependencies. GPL or AGPL requires legal approval before adoption.

3.5 Secrets Management

ID	Severity	Area	Requirement / Prohibition
U-S01	🔴 CRITICAL	Secrets in Code	Zero secrets in source code. Use environment variables or a secrets manager (Vault, AWS Secrets Manager, Azure Key Vault).
U-S02	🔴 CRITICAL	Git History	Run secret-scanning (git-secrets, truffleHog) in CI. Any leaked secret must be rotated immediately, not just deleted.
U-S03	🟠 MAJOR	Secret Rotation	Secrets must be rotatable without downtime. Document rotation procedures in the service runbook.

4. Language-Specific Standards

4.1 Java Spring Boot

Security-Critical Rules — Java Spring Boot

ID	Severity	Area	Requirement / Prohibition
J-S01	🔴 CRITICAL	Spring Security	Spring Security must be configured on every application. Never disable CSRF protection for stateful apps. Never use permitAll() on sensitive endpoints.

ID	Severity	Area	Requirement / Prohibition
J-S02	🔴 CRITICAL	Actuator	Disable or restrict all Spring Actuator endpoints in production. /actuator/env and /actuator/heapdump must never be publicly accessible.
J-S03	🔴 CRITICAL	Deserialization	Never deserialize untrusted data with Java native deserialization. Use Jackson with strict type handling. Disable default typing globally.
J-S04	🔴 CRITICAL	SSRF	Validate and whitelist all URLs before making outbound HTTP calls. Never pass raw user-supplied URLs to RestTemplate or WebClient.
J-M01	🟠 MAJOR	SQL (JPA)	Use Spring Data JPA named parameters or Criteria API. Never build JPQL or native queries via string concatenation.
J-M02	🟠 MAJOR	JWT	Validate JWT signature, expiry, issuer, and audience. Never use the 'none' algorithm. Store signing keys in a secrets manager.
J-M03	🟠 MAJOR	Error Handling	Return generic error messages to clients. Log full stack traces server-side only. Never expose internal paths or class names.
J-M04	🟠 MAJOR	Logging	Use SLF4J + Logback. Never log sensitive fields. Sanitize log inputs to prevent log injection (newline, ANSI escape).

Java Code Quality Requirements

- Use Java 17+ LTS. No active development on Java 8 or 11.
- Enable SpotBugs and Checkstyle in the Maven/Gradle build. Fail build on CRITICAL findings.
- All public APIs must have OpenAPI/Swagger annotations.
- Unit test coverage minimum: 80% line coverage. Integration tests for all API endpoints.
- Use @Validated for request body validation. Apply Bean Validation constraints at DTO level.

4.2 Python

Security-Critical Rules — Python

ID	Severity	Area	Requirement / Prohibition
P-S01	🔴 CRITICAL	Pickle / Eval	Never use pickle, marshal, or yaml.load() on untrusted data. Never use eval() or exec() on user input.
P-S02	🔴 CRITICAL	Command Execution	Never pass user input to os.system(), subprocess with shell=True, or popen. Use subprocess with a list and shell=False.

ID	Severity	Area	Requirement / Prohibition
P-S03	🔴 CRITICAL	SQL Injection	Use SQLAlchemy parameterized queries or ORM methods. Never use % or .format() to build SQL strings.
P-S04	🔴 CRITICAL	SSTI	Never render user-controlled strings directly in Jinja2 templates via Template(user_input).render(). Use the loader approach.
P-M01	🟠 MAJOR	Dependencies	Use pip-audit or Safety in CI. Pin all versions in requirements.txt. Use virtual environments — never system Python.
P-M02	🟠 MAJOR	Django/Flask	Enable CSRF middleware (Django). Set secure, httponly, samesite cookie flags. Use DEBUG=False in any non-local environment.
P-M03	🟠 MAJOR	Secrets	Use python-decouple or os.environ. Never use .env files committed to VCS. Use Vault or cloud secrets manager in production.
P-M04	🟠 MAJOR	Error Handling	Never expose tracebacks to end users. Use structured logging (structlog). Never log request bodies containing PII.

Python Code Quality Requirements

- Python 3.11+ required. No Python 2 or Python 3.8-.
- Use type hints on all function signatures. Run mypy in strict mode in CI.
- Linting: ruff or flake8 + black formatting. Build fails on linting errors.
- Run Bandit (SAST) in CI. Address all HIGH findings before merge.
- Use pytest. Minimum 80% coverage. Use pytest-cov to enforce in CI.

4.3 Go

Security-Critical Rules — Go

ID	Severity	Area	Requirement / Prohibition
G-S01	🔴 CRITICAL	SQL Injection	Use database/sql with ? or $N placeholders. Never use fmt.Sprintf() to build SQL queries.
G-S02	🔴 CRITICAL	Command Injection	Never pass user input to exec.Command with shell interpolation. Whitelist acceptable commands and arguments.
G-S03	🔴 CRITICAL	Path Traversal	Sanitize all file path inputs with filepath.Clean() and verify the result is within the expected base directory.

ID	Severity	Area	Requirement / Prohibition
G-S04	🔴 CRITICAL	TLS Config	Never set InsecureSkipVerify: true in tls.Config in non-test code. Use minimum TLS 1.2 and restrict cipher suites.
G-M01	🟠 MAJOR	Error Handling	Never silently discard errors with _. All errors must be handled, logged, or explicitly documented as intentionally ignored.
G-M02	🟠 MAJOR	Race Conditions	Run go test -race in CI. Protect all shared state with sync primitives. Never share memory between goroutines unsynchronized.
G-M03	🟠 MAJOR	Dependency Audit	Use govulncheck in CI. Run go mod tidy. Never vendor unverified third-party packages.
G-M04	🟠 MAJOR	HTTP Server	Set timeouts on all http.Server instances: ReadTimeout, WriteTimeout, IdleTimeout. Never use default http.ListenAndServe.

Go Code Quality Requirements

- Go 1.22+ required. Upgrade within 3 months of a new minor release.
- Run go vet, staticcheck, and golangci-lint in CI. Build fails on any error.
- All exported functions and types must have godoc comments.
- Use table-driven tests with testing package. Minimum 80% coverage enforced by CI.
- Use context.Context as first parameter in all functions that perform I/O or network calls.

4.4 Vue.js (Frontend)

Security-Critical Rules — Vue.js

ID	Severity	Area	Requirement / Prohibition
V-S01	🔴 CRITICAL	XSS via v-html	Never use v-html with user-controlled content. If unavoidable, sanitize with DOMPurify before binding.
V-S02	🔴 CRITICAL	Token Storage	Never store JWT or session tokens in localStorage. Use httpOnly cookies set by the backend or sessionStorage with short TTL.
V-S03	🔴 CRITICAL	CSRF	Attach CSRF tokens to all state-mutating API requests. Do not rely on CORS alone as a CSRF defense.
V-S04	🔴 CRITICAL	Open Redirect	Never redirect to a URL derived from user input without validating against an allowlist of internal routes.
V-M01	🟠 MAJOR	CSP	Configure a strict Content Security Policy header from the server. Avoid 'unsafe-inline' and 'unsafe-eval'.

ID	Severity	Area	Requirement / Prohibition
V-M02	🟠 MAJOR	Dependency Audit	Run npm audit --audit-level=high in CI. Address all HIGH and CRITICAL findings before merge.
V-M03	🟠 MAJOR	Sensitive Data	Never hard-code API keys, client secrets, or environment-specific URLs in frontend source code or .env files committed to VCS.
V-M04	🟠 MAJOR	Route Guards	All authenticated routes must have navigation guards. Never expose protected views by URL without server-side enforcement.

Vue.js Code Quality Requirements

- Vue 3 (Composition API) required. Vue 2 is end-of-life and prohibited for new services.
- Use TypeScript with strict mode. No any types without documented justification.
- ESLint with vue/recommended and @typescript-eslint/recommended. Prettier for formatting.
- Use Pinia for state management. No Vuex in new projects.
- Vitest for unit tests. Playwright or Cypress for E2E. Minimum 70% component coverage.

4.5 PHP

Security-Critical Rules — PHP

ID	Severity	Area	Requirement / Prohibition
PH-S01	🔴 CRITICAL	SQL Injection	Never use string interpolation to build SQL. Always use PDO or MySQLi prepared statements with bound parameters.
PH-S02	🔴 CRITICAL	Remote Code Exec	Never use eval(), preg_replace() with /e modifier, assert() with string arg, or create_function() on user input.
PH-S03	🔴 CRITICAL	File Inclusion	Never pass user input to include, require, include_once, or require_once. Use a static allowlist for all file inclusion.
PH-S04	🔴 CRITICAL	Deserialization	Never call unserialize() on untrusted data. Use JSON for data exchange. If unavoidable, implement __wakeup() and __unserialize() guards.
PH-S05	🔴 CRITICAL	Command Injection	Never pass user input to shell_exec(), exec(), system(), passthru(), or backtick operator. Whitelist all OS interactions.
PH-M01	🟠 MAJOR	XSS	Always use htmlspecialchars($val, ENT_QUOTES, UTF-8) when outputting user data in HTML. Use a templating engine with auto-escaping (Twig, Blade).

ID	Severity	Area	Requirement / Prohibition
PH-M02	🟠 MAJOR	Session Security	Use session_regenerate_id(true) after login. Set session.cookie_httponly, session.cookie_secure, and session.cookie_samesite in php.ini.
PH-M03	🟠 MAJOR	File Upload	Validate MIME type using finfo, not $_FILES type. Rename uploaded files server-side. Store outside webroot. Never execute uploads.
PH-M04	🟠 MAJOR	Error Handling	Set display_errors=Off and log_errors=On in production. Never expose stack traces, file paths, or DB credentials in HTTP responses.
PH-M05	🟠 MAJOR	Dependency Audit	Use Composer with composer.lock committed. Run composer audit in CI. No packages with known Critical/Major CVEs.

PHP Code Quality Requirements

- PHP 8.2+ required. PHP 7.x and below are end-of-life and prohibited.
- Enforce strict_types=1 at the top of every file. Use typed properties and return types throughout.
- Use PHP-CS-Fixer or PHP_CodeSniffer (PSR-12 standard). Run in CI; build fails on violations.
- Run PHPStan at level 8 (or Psalm equivalent) in CI. Address all errors before merge.
- Use PHPUnit for testing. Minimum 80% line coverage. Mock external I/O in unit tests.
- Use a modern framework (Laravel 10+, Symfony 6+). No raw PHP applications for new services.

4.6 Angular (Frontend)

Security-Critical Rules — Angular

ID	Severity	Area	Requirement / Prohibition
A-S01	🔴 CRITICAL	XSS / innerHTML	Never bind user data to [innerHTML] or call bypassSecurityTrustHtml() without explicit DOMPurify sanitization. These methods disable Angular built-in XSS protection.
A-S02	🔴 CRITICAL	Token Storage	Never store JWT or session tokens in localStorage or sessionStorage. Use httpOnly, Secure, SameSite=Strict cookies managed by the backend.
A-S03	🔴 CRITICAL	Open Redirect	Never use Router.navigate() or window.location with unvalidated user input. Validate all redirect targets against an internal route allowlist.

ID	Severity	Area	Requirement / Prohibition
A-S04	🔴 CRITICAL	**Template Injection**	Never construct Angular templates dynamically from user input. Never use JIT compilation in production. Always use AOT (Ahead-of-Time) compilation.
A-M01	🟠 MAJOR	**CSRF**	Use Angular HttpClient which reads XSRF-TOKEN cookie automatically. Verify the backend sets and validates the CSRF cookie/header pair correctly.
A-M02	🟠 MAJOR	**Route Guards**	Implement CanActivate and CanActivateChild guards for all authenticated routes. Never rely solely on hiding navigation elements as an access control.
A-M03	🟠 MAJOR	**HTTP Interceptors**	Use HTTP interceptors to attach auth headers. Never hard-code Authorization tokens in service files or environment configs committed to VCS.
A-M04	🟠 MAJOR	**CSP**	Configure a strict Content Security Policy from the server. Angular AOT removes the need for unsafe-eval — do not add it. Avoid unsafe-inline.
A-M05	🟠 MAJOR	**Dependency Audit**	Run npm audit --audit-level=high in CI. Keep Angular and peer dependencies on an active LTS version. No unmaintained packages.

Angular Code Quality Requirements

- Angular 17+ (standalone components) required for new applications. Upgrade to active LTS within 6 months of a major release.
- TypeScript strict mode enabled (strict: true in tsconfig.json). No any types without justified and documented exceptions.
- Use ESLint with @angular-eslint/recommended ruleset. Prettier for formatting. Build fails on lint errors.
- Always build with AOT (ng build --configuration production). Never deploy JIT-compiled bundles.
- Use Jest or Karma + Jasmine for unit tests. Cypress or Playwright for E2E. Minimum 70% component and service coverage.
- Use Angular reactive forms (not template-driven) for all form handling that involves server submission.

5. Code Review & PR Gate Requirements

No pull request may be merged unless all of the following gates are satisfied. Gates are enforced by CI pipeline configuration and are not bypassable without Director approval.

5.1 Mandatory CI Gates

#	Gate	Applies To	Failure Action
1	SAST scan — zero Critical / Major findings	All	**Block merge**
2	Dependency vulnerability scan — zero Critical findings	All	**Block merge**
3	Secret scan — zero secrets detected in diff	All	**Block merge**
4	Unit test suite passes	All	**Block merge**
5	Code coverage ≥ threshold (Java/Python/Go/PHP: 80%, Vue/Angular: 70%)	All	**Block merge**
6	Linting / static analysis passes with zero errors	All	**Block merge**
7	At least 2 peer approvals from engineers at same or higher level	All	**Block merge**
8	Security-relevant changes require 1 additional approval from Security Lead	Security scope	**Block merge**

5.2 Code Review Checklist

Reviewers must verify the following during every review:

- Input validation is present for all untrusted data sources (request body, query params, headers, files).
- Authentication and authorization are enforced — no implicit public access.
- No secrets, credentials, or PII in code, comments, or test fixtures.
- Error handling does not leak internal details to the client.
- Logging is structured, non-sensitive, and does not expose PII.
- New dependencies are justified, licensed, and free of known Critical/Major CVEs.
- Database queries use parameterized statements — no string concatenation.
- Tests cover the happy path, edge cases, and at least one failure/error path.

6. Branching, Environments & Deployment

6.1 Branching Model

- main — protected. No direct pushes. Deploys to production.
- develop — integration branch. Deploys to staging automatically.
- feature/<ticket-id>-<short-description> — developer branches.
- hotfix/<ticket-id>-<short-description> — merged to main and back-merged to develop.
- release/<version> — created from develop for release preparation.

6.2 Environment Rules

Environment	Allowed Actions	Prohibited
Development (local)	Any — engineers own their environment.	Production secrets, real PII.
Staging	Integration testing, QA, performance testing.	Production data, bypassing CI gates.
Production	Automated deployments from main only.	Manual code changes, debug mode, test data.

7. Security Incidents & Exception Process

7.1 Security Incident Response

Any engineer who discovers a potential security vulnerability — in code, infrastructure, or a third-party component — must:

- Report immediately to the Software Engineering Director and the Security Lead via the designated secure channel (do not use public Slack channels or email for disclosure).
- Do not attempt to exploit or verify the vulnerability in a production environment.
- Open a confidential security ticket with reproduction steps, affected components, and severity assessment.
- For Critical findings: acknowledge within 1 hour, patch within 24 hours.
- For Major findings: acknowledge within 4 hours, patch within 72 hours.

7.2 Exception Process

Exceptions to this standard are rare and require formal approval. The process is:

- Engineer submits an Exception Request Form with: rule ID, business justification, compensating controls, and risk owner.
- Security Lead reviews and provides a recommendation within 3 business days.
- Software Engineering Director grants or denies the exception. All exceptions are time-bound (maximum 90 days) and logged.
- No exception may be granted for CRITICAL-severity rules without Director sign-off and a written risk acceptance.

8. Compliance Summary

Engineers are accountable for the security and quality of the code they author and approve.

You Must	You Must Never
• Run all CI gates before requesting review • Use parameterized queries for all DB access • Store secrets in approved secrets managers • Validate all inputs server-side • Use TLS 1.2+ for all communications • Report security findings immediately • Write tests covering failure and edge cases	✕ Commit secrets or credentials to VCS ✕ Concatenate user input into SQL/OS commands ✕ Bypass or disable CI security gates ✕ Use deprecated cryptographic algorithms ✕ Ship dependencies with Critical/Major CVEs ✕ Expose stack traces or internal errors to users ✕ Merge PRs that fail security or test gates

Questions? Contact the Software Engineering Director